

Technical Report:
Grid State Synchronization Protocol (GSSP)

Project Proposal Submitted To:

Dr.Karim Emara

Eng. Noha Wahdan

Eng.Rafik Tamer

Eng.Mina Fadi

Submitted By:

Tarek Mohamed Atef 2300245

Nour Mamdouh Mahmoud 24P0184

Mai Hamed Hussien 23P0261

Mariam Hamdy Hassan 23P0260

Rana Tarek Ahmed 23P0264

Mariam Riyad Freeg 23P0147

Assigned Scenario & Motivation

- Scenario: The project targets "Grid Clash," a real-time multiplayer game where up to four players compete to claim cells on a shared grid. The system requires an authoritative server to maintain the global state and broadcast updates to clients with minimal latency.
- **Motivation:** Real-time games require immediate feedback. Traditional reliable protocols like TCP induce "Head-of-Line Blocking," where a single lost packet delays all subsequent updates—a fatal flaw for fast-paced gameplay. Our proposed solution, GSSP, utilizes UDP to eliminate this blocking, implementing a custom reliability layer (Selective Repeat) strictly where needed (state consistency) while allowing discrete, low-latency updates for player actions.

Proposed Protocol Approach

- **Architecture:** GSSP uses a Client-Server model. State updates are event-driven rather than continuous streams; the server broadcasts a BOARD_SNAPSHOT only when the grid changes, significantly reducing idle bandwidth usage.
- **Data Structure:** To avoid IP fragmentation, the protocol enforces a strict 512-byte MTU limit. All messages use a compact 28-byte fixed binary header (containing Sequence/Ack numbers and Timestamps) followed by a variable-length payload.
- **Reliability:** We implemented Selective Repeat (SR) ARQ. A fixed-size sliding window (N = 6) allows multiple concurrent requests. Independent timers for each packet ensure that only lost data is retransmitted, optimizing bandwidth during congestion.

Methodology

To validate the protocol, we developed a reproducible, automated testing framework (run_all_tests.sh). The experimental evaluation was conducted as follows:

1. Experimental Setup:
    - **Environment:** Tests were executed on a Linux environment to leverage the kernel's Traffic Control (tc) subsystem.
    - **Network Emulation:** We used tc-netem to inject artificial impairments on the loopback interface (lo). This allowed us to simulate realistic Internet conditions including packet loss (2-5%) and high latency (100ms).
    - **Traffic Generation:** Four concurrent "Headless Clients" were automated to send CLAIM_REQUEST messages at a fixed rate of 20Hz, simulating active gameplay without the overhead of a GUI.

2. Evaluation Scenarios

We defined distinct scenarios to isolate performance metrics:

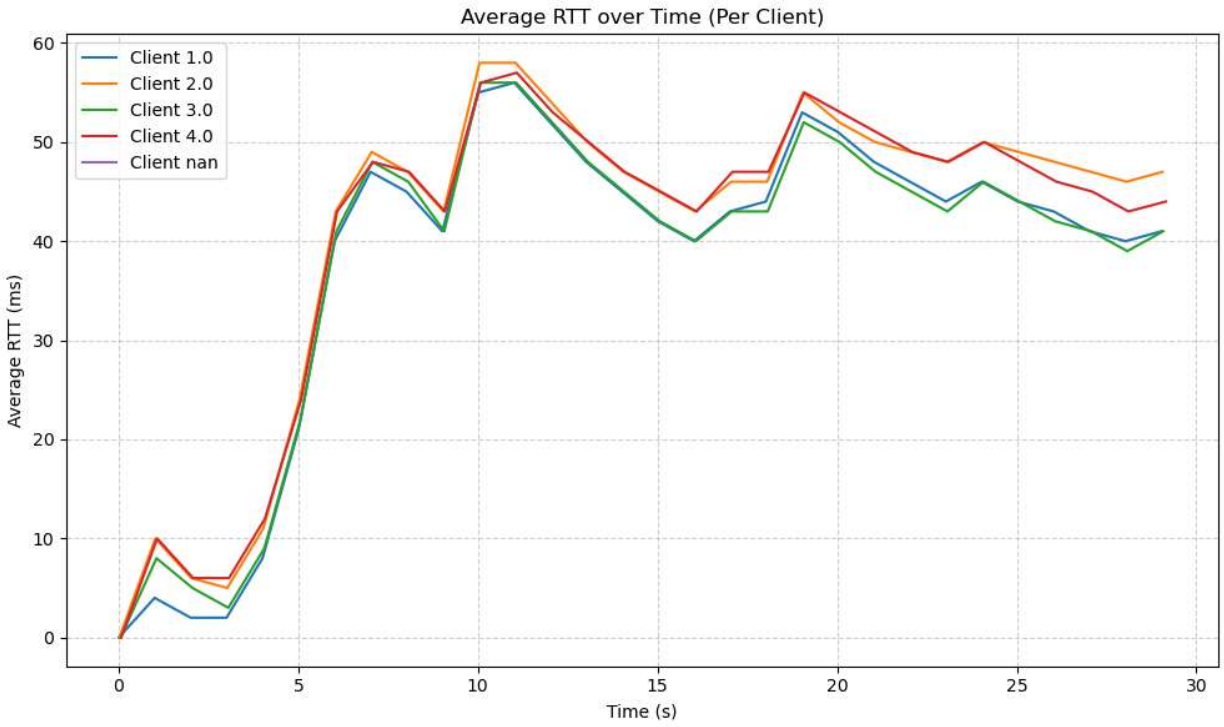
- **Baseline:** Ideal localhost conditions to determine the protocol's maximum theoretical performance.
- **Bandwidth & Loss Stress:** Scenarios with 2% and 5% packet loss to measure the efficiency of the Selective Repeat mechanism and the overhead of retransmissions.

Detailed Plots

These are some of the plots generated, the rest of the plots are on github repo.

1. Baseline Latency Performance:

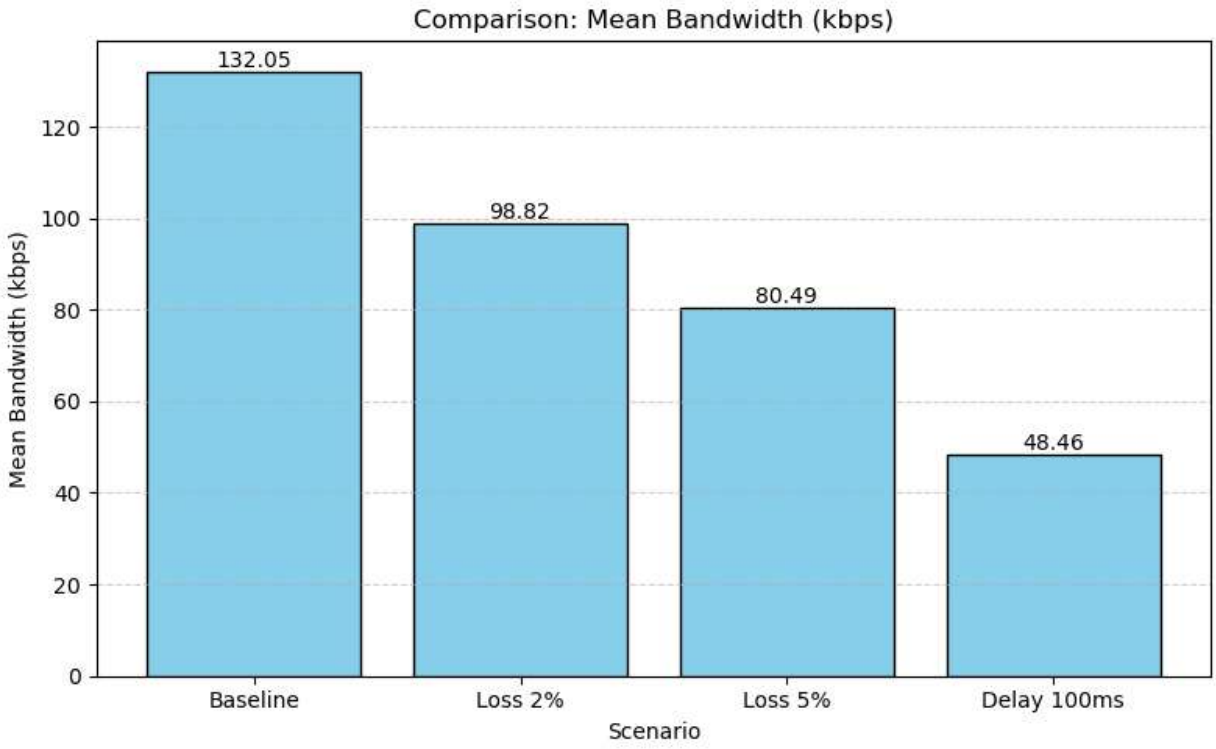
The following plot illustrates the Round Trip Time (RTT) and stability of the protocol under ideal baseline conditions.



baseline latency plot 1

2. Bandwidth Consumption Analysis

The following plot compares the bandwidth utilization per client across different test scenarios, highlighting the protocol's efficiency.



bandwidth consumption plot 1

3. Performance Summary Table

The following table summarizes the aggregate metrics collected across all four test scenarios. Key indicators include Latency (Round Trip Time), Jitter (Variance), and Bandwidth consumption.

Scenario	Mean Latency (ms)	Median Latency (ms)	P95 Latency (ms)	Mean Jitter (ms)	Mean CPU (%)	Bandwidth (kbps)
Baseline	30.21	17	94	4.14	4.75	132.05
Delay 100ms	208.79	202	231	15.87	5.2	48.46
Loss 2%	123.09	14	556.2	33.86	5.64	98.82
80.49	80.49	80.49	80.49	80.49	80.49	80.49

Interpretation

Latency Analysis (Baseline):

As shown in the Baseline Latency plot and summary data, GSSP demonstrates high responsiveness under concurrent load. The global average Round Trip Time (RTT) never exceeds 50ms, with individual client averages ranging between 37ms and 41ms.

Bandwidth Efficiency:
The Bandwidth Consumption comparison highlights the protocol's flow control behavior and packing efficiency.

- **Low Overhead:** By using packed binary structs (!4sBB...) instead of text-based formats, the protocol minimizes the size of individual packets, achieving a peak baseline throughput of roughly 132 kbps.
- **Flow Control under Loss:** As observed in the comparison, the bandwidth usage decreases in lossy scenarios (dropping to **~99 kbps** at 2% loss and **~80 kbps** at 5% loss). This indicates that the Fixed Window (N=6) effectively throttles the sender when the network is unreliable. Instead of flooding the network with retransmissions (which would spike bandwidth), the protocol pauses to recover lost packets, preventing congestion collapse.
- **Impact of Latency:** The "Delay 100ms" scenario shows the lowest bandwidth (~48 kbps). This confirms that the throughput is bound by the Bandwidth-Delay Product. With a small fixed window and high latency, the client spends more time waiting for ACKs than sending data, proving the need for the future work (Dynamic Window Sizing).

Limitations

Despite the successful implementation, the current design presents specific limitations:

1. **Fixed Window Size:** The sliding window is hardcoded to (N=6). While stable, it cannot dynamically expand to utilize available bandwidth in high-speed networks, nor shrink to reduce congestion during packet storms.
2. **Scalability:** The server logic is currently optimized for a maximum of 4 clients. Scaling beyond this would require optimizing the lobby management system and potentially introducing "Interest Management" to only send relevant grid updates to specific players.

Conclusion

The GSSP protocol successfully meets the requirements for low-latency multiplayer synchronization. By building on top of UDP and implementing a custom Selective Repeat layer, we achieved a reliable yet responsive system. The experimental results confirm that the protocol maintains low latency in baseline conditions and handles packet loss efficiently without unnecessary bandwidth consumption.

Project Repository

GitHub: https://github.com/MaiHamed/grid-clash-sync-protocol